UNIVERSITÉ ABDELMALEK ESSAÂDI

École Nationale des Sciences Appliquées de Tétouan

Rapport de livraison

Séances 3 & 4 — Modélisation JPA et CRUD REST

HireLink

Plateforme ATS de recrutement (candidat · recruteur · admin)

Mise à jour du cahier des charges · Diagramme de classes · Réponses aux requis

Étudiante	Douae EL ASSAL
Formation	GI3 — ENSA Tétouan
Module	Frameworks Technologie Web
Stack	Spring Boot 3 · Java · JPA/Hibernate · MySQL · HTML/JS
API	`http://127.0.0.1:8081/api/v1`
Frontend	`http://127.0.0.1:5500`
Base	`hirelink` (MariaDB / phpMyAdmin)
Date	octobre 2026

Ce document aligne le cahier des charges et les livrables S3/S4 sur l'état réel de l'application.

Table des matières

1 Cahier des charges — version actualisée **2**
- 1.1 Contexte et problème . . . 2
- 1.2 Acteurs . . . 2
- 1.3 Fonctionnalités livrées (état actuel de l'app) . . . 2
  - 1.3.1 Espace candidat . . . 2
  - 1.3.2 Espace recruteur . . . 3
  - 1.3.3 Espace admin . . . 3
  - 1.3.4 Sécurité et confiance . . . 3
- 1.4 Règles métier clés . . . 3
- 1.5 Évolutions par rapport aux versions CDC antérieures . . . 3

2 Architecture et modèle de données **4**
- 2.1 Architecture en couches . . . 4
- 2.2 Diagramme de classes (état actuel) . . . 4
- 2.3 Entités et tables . . . 5
- 2.4 Relations JPA principales . . . 5

3 Ce que nous avons réalisé **6**
- 3.1 Séance 3 — Modélisation JPA / Hibernate . . . 6
- 3.2 Séance 4 — CRUD et API REST . . . 6
- 3.3 Compléments livrés (hors « 3 CRUD » mais cohérents) . . . 6
- 3.4 Score CV, synthèse et vérification (clarifications produit) . . . 6
  - 3.4.1 Score CV . . . 6
  - 3.4.2 Analyse vs synthèse . . . 7
  - 3.4.3 Vérification entreprise . . . 7

4 Réponses aux requis — Séance 3 **8**

5 Réponses aux requis — Séance 4 **9**

6 Annexes **10**
- 6.1 Comptes de démonstration . . . 10
- 6.2 Démarrage rapide . . . 10
- 6.3 Chemins utiles . . . 10
- 6.4 Conclusion . . . 10

Chapitre 1

Cahier des charges — version actualisée

1.1 Contexte et problème

HireLink n'est pas un simple jobboard. C'est un **ATS** (Applicant Tracking System) qui orchestre le cycle de sélection après candidature : pipeline d'étapes, délai de première réponse affiché (SLA), motifs de refus documentés, matching compétences, scorecard, et vérification d'entreprise avant publication.

Trois refus du marché motivent le produit :

— offres sans engagement de délai de réponse ;

— refus sans motif exploitable ;

— entreprises non authentifiées qui publiquent librement.

1.2 Acteurs

Acteur	Rôle dans HireLink
Candidat	Dépose un CV, complète le profil, consulte offres et entreprises, postule, suit ses dossiers, utilise la synthèse CV (crédits).
Recruteur	Crée une organisation (ICE), obtient un badge (auto ou admin), publie des offres, gère le pipeline, invite l'équipe.
Administrateur	Suit les stats, vérifie les entreprises en attente, gère plans / comptes / motifs de refus.

1.3 Fonctionnalités livrées (état actuel de l'app)

1.3.1 Espace candidat

— Tableau de bord (profil, score CV, prévisualisation CV, offres recommandées, Premium).

— Offres (liste / détail), entreprises avec logo et badge, mes candidatures.

— Paramètres : Profil, Parcours, Image (sans photo stockée), CV, alertes ; formule **Essentiel** / **Performance**.

— Topbar : Passer Premium · cloche notifications · icône profil.

— Police UI : **Inter** ; palette bordeaux HireLink.

1.3.2 Espace recruteur

— Dashboard, pipeline (SLA), CRUD offres, fiche entreprise (ICE, logo, relancer vérification), paramètres / Premium.

— Publication bloquée si l'entreprise n'est pas vérifiée.

1.3.3 Espace admin

— Stats, file « à vérifier », bouton Vérifier / Retirer, plans, comptes.

1.3.4 Sécurité et confiance

— Auth : e-mail + mot de passe, OTP à l'inscription, Google Sign-In.

— Vérification entreprise hybride (ICE + e-mail pro → auto ; sinon revue admin).

— JWT ; rôles `CANDIDAT`, `RECRUTEUR`, `ADMINISTRATEUR`.

1.4 Règles métier clés

1. Inscription libre ≠ publication libre : badge entreprise obligatoire pour publier.
2. Chaque offre affiche un délai de 1re réponse (SLA).
3. Un refus documenté s'appuie sur le catalogue de motifs.
4. L'IA (synthèse CV) assiste ; elle ne postule ni ne décide.
5. Essentiel candidat : 5 synthèses CV / mois ; Performance : sans plafond.

1.5 Évolutions par rapport aux versions CDC antérieures

Point	Mise à jour
UX candidat	Alignée sur une expérience type Stagiaires (dashboard 3 colonnes, entreprises, candidatures split).
Organisation	Champs logo, site, secteur, ville, présentation, `verificationMethod`.
Vérification	Automatique (registre ICE) + file admin ; API `/organizations/public`, `/mine/verify`.
Typographie	Inter pour toute l'UI.
Formules	Performance / Premium décrites (bénéfices, prix, retour possible).

Chapitre 2

Architecture et modèle de données

2.1 Architecture en couches

Frontend HTML/JS $\xrightarrow{\text{JWT}}$ Controller $\rightarrow$ Service $\rightarrow$ Repository JPA $\rightarrow$ MySQL `hirelink`

La source de vérité du schéma est le modèle Java (`@Entity`). Hibernate applique `ddl-auto=update` au démarrage. phpMyAdmin sert à contrôler, pas à inventer le schéma.

2.2 Diagramme de classes (état actuel)

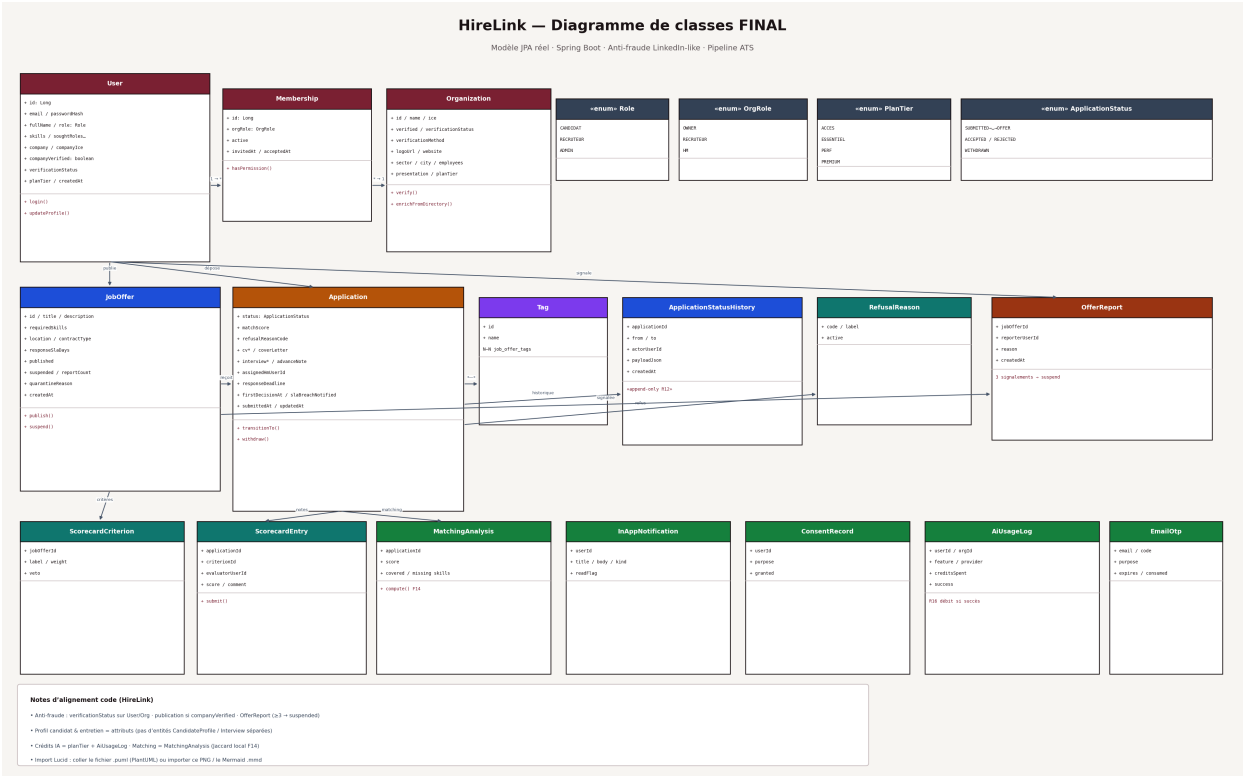


Figure 2.1 – Diagramme de classes HireLink (JPA réel : User, Organization, Membership, JobOffer, Application, Tag, Notification, Scorecard, Matching, vérif entreprise, etc.).

2.3 Entités et tables

Entité	Table	Rôle
User	users	Comptes, profil, CV, ICE, plan
Organization	organizations	Entreprise, badge, logo, plan
Membership	memberships	Lien User↔Org + OrgRole
JobOffer	job_offers	Offres + SLA + quarantine
Application	applications	Candidatures + statut pipeline
Tag	tags / job_offer_tags	Tags ManyToMany
RefusalReason	refusal_reason_catalog	Motifs de refus
InAppNotification	notifications	Cloche in-app
MatchingAnalysis	matching_analyses	Scores matching
Scorecard*	scorecard_*	Évaluation multi-critères
AiUsageLog	ai_usage_logs	Crédits / synthèse CV

2.4 Relations JPA principales

— User (recruteur) 1—∗ JobOffer ; User (candidat) 1—∗ Application.

— JobOffer 1—∗ Application.

— User ↔ Organization via Membership (association avec attributs OWNER / RECRUTEUR / HIRING_MANAGER).

— JobOffer ↔ Tag : `@ManyToMany`.

— Organization → User (owner) : `@ManyToOne`.

Chapitre 3

Ce que nous avons réalisé

3.1 Séance 3 — Modélisation JPA / Hibernate

1. Identification des entités à partir du CDC ATS.
2. Annotations `@Entity`, enums (`Role`, `OrgRole`, `ApplicationStatus`, `PlanTier`).
3. Mapping des relations (ci-dessus).
4. Configuration datasource `127.0.0.1:3306/hirelink` + `ddl-auto=update`.
5. Contrôle phpMyAdmin + diagramme de classes livré.
6. Formulaires frontend alignés (profil, offres, pipeline, entreprise).

3.2 Séance 4 — CRUD et API REST

Trois CRUD complets démontrés :

Entité	**Endpoints**
JobOffer	POST/GET/PUT/DELETE `/api/v1/offers` (+ `/mine`, `/public`)
RefusalReason	CRUD `/api/v1/refusal-reasons` (admin)
InAppNotification	CRUD + `/{id}/read` `/api/v1/notifications`

Preuves : UI `recruiter-offers.html`, `admin-refusal-reasons.html`, cloche ; collection Postman `docs/postman/HireLink_CRUD_Seance4.postman_collection.json`.

3.3 Compléments livrés (hors « 3 CRUD » mais cohérents)

Auth (OTP / Google / complete-profile), pipeline candidatures, organisations (vérif + logos publics), admin entreprises, CV multipart + synthèse `/ai/cv-summary`, marketplace d'offres seedées.

3.4 Score CV, synthèse et vérification (clarifications produit)

3.4.1 Score CV

Calcul côté frontend sur les **sections** du CV : Identité /15, Expériences /25, Formation /20, Compétences /25, Langues /15. Seuils : A ≥ 80, B ≥ 55, sinon C. Ce n'est pas de l'IA.

3.4.2 Analyse vs synthèse

L'analyse (onglets) est une checklist automatique. La synthèse (`POST /ai/cv-summary`) produit un texte court à partir des sections ; elle consomme un crédit Essentiel (5/mois) et est journalisée dans `ai_usage_logs`.

3.4.3 Vérification entreprise

ICE + nom + e-mail professionnel → badge automatique si registre OK ; sinon file admin. Sans badge : publication refusée.

Chapitre 4

Réponses aux requis — Séance 3

Requis Séance 3

Construire le modèle de données et le relier au backend : entités JPA, relations, Hibernate, MySQL, diagramme de classes, formulaires alignés.

Réponse — statut : SATISFAIT

1. **Entités JPA** : package `ma.ensat.hirelink.model` (User, Organization, Membership, JobOffer, Application, Tag, RefusalReason, InAppNotification, Scorecard, Matching, etc.).
2. **Relations** : `@ManyToOne`/`@OneToMany`, Membership N–N, `@ManyToMany` JobOffer↔Tag.
3. **Hibernate** : `spring.jpa.hibernate.ddl-auto=update` — tables créées/mises à jour au démarrage Spring Boot (pas de `schema.sql` comme source de vérité).
4. **MySQL** : schéma `hirelink` visible dans phpMyAdmin (`http://127.0.0.1/phpmyadmin/`).
5. **Diagramme de classes** : figure 2.1 et fichiers `docs/diagrammes/diagramme_classes_hirelink_FINAL.*`.
6. **Formulaires** : pages candidat / recruteur / admin branchées sur les champs persistés.

Compétences couvertes : JPA, Hibernate, ORM, mapping objet-relationnel, relations, persistance.

Chapitre 5

Réponses aux requis — Séance 4

Requis Séance 4

Développer un CRUD complet sur au moins trois entités, exposer une API REST, préparer les interfaces frontend, tester avec Postman.

Réponse — statut : SATISFAIT

1. **CRUD 1 — JobOffer** : Controller `/api/v1/offers` + Service + Repository ; UI recruteur créer / modifier / supprimer ; gate entreprise vérifiée.
2. **CRUD 2 — RefusalReason** : endpoints admin ; UI `admin-refusal-reasons.html`.
3. **CRUD 3 — InAppNotification** : create / list / mark-read / delete ; cloche dans la topbar.
4. **Architecture** : Controller → Service → Repository → Entity (démontrable ligne par ligne).
5. **Frontend** : `fetch` + JWT (`frontend/js/api.js`) vers `http://127.0.0.1:8081/api/v1`.
6. **Postman** : collection officielle dans `docs/postman/` (login JWT puis scénarios CRUD).

Compétences couvertes : Spring Data, Repository, Service, REST Controller, CRUD HTTP, manipulation de données.

Tableau récapitulatif « demandé / livré »

Demande	Livré
Modèle JPA + relations	Entities + Membership + ManyToMany Tags
Hibernate → MySQL	`ddl-auto=update`, base `hirelink`
Diagramme de classes	PNG + PlantUML + figure dans ce rapport
CRUD ≥ 3 entités	Offres, Motifs, Notifications
API REST	Controllers `/api/v1/**`
UI + Postman	Pages app + collection Postman

Chapitre 6

Annexes

6.1 Comptes de démonstration

— Admin : `admin@hirelink.ma` / `Admin123!`

— Recruteur : `n.bennani@atlasdigital.ma` / `Recruteur123!`

— Candidat : `candidat@hirelink.ma` / `Candidat123!`

6.2 Démarrage rapide

1. XAMPP : MySQL + Apache (phpMyAdmin).
2. Backend : `mvn spring-boot:run` dans `backend/` (port 8081).
3. Frontend : `python -m http.server 5500` dans `frontend/`.
4. Santé : `GET /api/v1/health` → UP + `catalog=hirelink`.

6.3 Chemins utiles

```
HireLink/
  backend/src/main/java/ma/ensat/hirelink/{model,repository,service,controller}
  frontend/app/
  docs/diagrammes/diagramme_classes_hirelink_FINAL.png
  docs/postman/HireLink_CRUD_Seance4.postman_collection.json
  docs/cahier-des-charges/
  docs/livraison-seance3-4/rapport-latex/   ← sources de ce PDF
```

6.4 Conclusion

Les objectifs des séances 3 et 4 sont **atteints** : modèle JPA cohérent avec le métier ATS, schéma MySQL généré par Hibernate, CRUD REST démontrable, UI et Postman. Le cahier des charges et le diagramme de classes de ce document reflètent l'**état actuel** de HireLink (vérification entreprise, logos, UX candidat, formules, Inter).